LLD Practice Platform

A small Spring Boot app where a learner practises Low-Level Design on three problems (Parking Lot, Elevator, Vending Machine), submits Java code, and gets feedback that separates **rule-based facts** from **AI judgment**.

The loop: choose a problem, design, submit, get feedback, review, try again. A change request for each problem measures how much of the learner's own design had to move.

Run it

Requires JDK 17 or newer (built and tested on 21). Nothing else to install; Maven is bundled as `mvnw`.

```
./mvnw spring-boot:run          # Windows: mvnw.cmd spring-boot:run
```

Open http://localhost:8080. Attempts are stored in `./data/lld-practice.mv.db` and survive a restart.

The AI review

Set `GEMINI_API_KEY` to use the real review (optional `GEMINI_MODEL`, default `gemini-2.5-flash`).

Without a key the app runs in **demo mode**: the AI stage returns a single placeholder note that says so. Everything else (parsing, the seven rules, the diagram, history, the change round) is real. Demo mode exists so the whole flow can be tried with no account.

Test it

```
./mvnw test
```

158 tests (one is the opt-in live review check, skipped unless `GEMINI_API_KEY` is set). They cover the parser, each rule, the failure paths (timeout, malformed JSON, invented classes, nothing parses, a throwing rule, a restart mid-evaluation, retry pressed twice), the change round, the history diff, the persistence round trip, the HTTP flow including HTML escaping, and an architecture test that enforces the package dependency rules. Tests use a scripted `FakeLlmClient`, so none of them call a real model.

What it does

Layer	What the learner sees
FACT	Seven deterministic rules on the structure of their code, available immediately. Labelled `FACT`.
JUDGMENT	One LLM review, run in the background. Labelled "AI opinion, advisory". Findings that cite classes or methods not in the code are dropped.
Change round	Edit the same code for the problem's change request. Shown as added / modified / removed / interfaces touched, with an ADDITIVE, CONTAINED or SPREAD verdict.

Layer	What the learner sees
History	Resolved / new / persisting findings against the previous attempt, matched by fingerprint.

Attempt states: `DRAFT → SUBMITTED → EVALUATING → COMPLETED | PARTIAL | FAILED`. If the AI stage fails after retries the attempt ends `PARTIAL`: the facts stay and a Retry button appears. If nothing in the submission parses it ends `FAILED` with the parse errors.

Limits you should know about

- **Code is parsed, never compiled or run.** Compile errors, missing imports and unknown types are ignored. Behaviour is not tested.
- **Java only**, top-level types only. Nested types are skipped with a visible warning.
- The AI review sees class names, signatures and relationships, **not method bodies**.
- Rule thresholds (for example "more than 12 methods") are starting defaults and have not been tuned against real learner submissions. They live in `RuleThresholds`.
- `CAPABILITY_OWNERSHIP` matches name fragments, so it can miss an unconventional name.
- The blast-radius cut-offs (CONTAINED = at most 2 touched and not more than added) are heuristics, configurable in `EvaluationProperties`.
- A learner is a cookie, not an account. Clearing cookies loses access to earlier attempts in the UI (the data stays in the database).
- The real Gemini call was run once end to end (model `gemini-flash-latest`, Parking Lot skeleton): it returned 3 findings that passed validation. That is one smoke run, not an evaluation of review quality.
- The class diagram is drawn in the browser by Mermaid (served from a WebJar, no CDN). It was checked in Chrome on the dark theme (it follows the system light/dark setting). The Mermaid source is always shown in a collapsed block as a fallback. Field types keep their generics in the diagram (`List~Elevator~`).
- History times are shown in UTC.
- While the AI review runs the page shows a spinner and an elapsed-seconds counter; buttons that trigger slow actions show a busy state.
- `CAPABILITY_OWNERSHIP` is a name-fragment heuristic. When no method is named after a capability it also looks for the same words in enum constants, field names and class names. If it finds them it reports MEDIUM (implied, give it an explicit method) instead of HIGH. If the design never uses those words at all, HIGH remains.

Layout

```
model/          immutable domain records (depend on nothing)
problem/        Problem, YAML-backed ProblemRepository
evaluation/     Evaluator, Rule ports; RuleEvaluator; evaluation/rules/ the seven rules
parser/         SubmissionParser port, ParserRegistry; parser/javacode/ JavaParser adapter
llm/            LlmClient port; prompt, reply parser, FindingValidator, review evaluator; Gemini, c
attempt/        Attempt, AttemptLifecycle (transition table), AttemptRepository port
orchestration/  services: AttemptService, EvaluationOrchestrator, ChangeImpactCalculator, FindingDi
persistence/    JPA adapter (submission and report stored as JSON columns)
web/            Thymeleaf controllers
src/main/resources/problems/   one YAML file per problem
```

Adding a problem means adding a YAML file. Adding a rule means adding a `Rule` class. Adding an evaluation approach means adding an `Evaluator`. Adding a submission format means adding a `SubmissionParser`. Spring picks each up as a list, so nothing else changes.

How good is the AI review? See docs/REVIEW_EVALUATION.md. Re-run it with `./mvnw test -Dtest=LiveReviewQualityTest` and a `GEMINI_API_KEY`.

More detail: docs/DESIGN_NOTE.md, docs/RESEARCH_NOTE.md, AI_USAGE.md.

AI usage

This file records decisions where the suggestion was not simply the spec, what was accepted or changed, and why.

1. Mermaid WebJar version

- **AI suggested:** pin `org.webjars.npm:mermaid:10.9.1` to serve Mermaid locally, as the spec requires (no CDN).
- **What happened:** that version does not exist on Maven Central, and the first version tried afterwards failed on conflicting transitive version ranges.
- **Decision:** use `11.12.3` and exclude all transitive dependencies, since `mermaid.min.js` is self-contained. A test checks the file is served.
- **Why:** the spec's offline requirement outweighs the cost of one pinned dependency. **Not verified:** the diagram drawing in a real browser (no browser was available during the build).

2. Finding fingerprint

- **Spec said:** fingerprint = rule id + subject.
- **AI suggested:** add an optional discriminator.
- **Why:** `CAPABILITY_OWNERSHIP` reports one finding per missing capability, all on the entry type. With the plain fingerprint, two different missing capabilities collapse into one in the resolved / new / persisting diff, hiding real progress. A test (`twoMissingCapabilitiesHaveDifferentFingerprints`) fails without the change.
- **Decision:** accepted. Other findings keep the exact spec fingerprint.

3. What "restates a FACT" means

- **Spec said:** drop an LLM finding that restates a FACT on the same subject.
- **AI first read it as:** drop any LLM finding on a subject that has any FACT.
- **Rejected that reading:** a class flagged for one reason can still deserve different AI comments (for example an encapsulation concern on a class flagged as an orphan).
- **Decision:** drop only when the FACT is from a related rule, using a small category-to-rule table in `FindingValidator`. Tested.

4. How much of a broken file to keep

- **Problem:** JavaParser may return a partial tree even for class-level syntax errors, which would give the learner misleading findings.

- **AI suggested:** keep a file with errors only when the errors are statement-level (JavaParser produces an unparsable-statement node); otherwise skip the whole file with a visible warning.
- **Why:** matches the spec's failure table (method-body error keeps structure, class-level error skips the file) and was checked with tests for both cases.

5. Demo mode for the AI stage

- **AI suggested:** when no `GEMINI_API_KEY` is set, use a client that returns one clearly labelled placeholder finding, instead of failing every attempt as PARTIAL.
- **Why:** lets a reviewer run the whole loop with no account. The placeholder says it is demo mode, so it cannot be mistaken for a real review.
- **Risk to check:** a reviewer must not read demo output as a real evaluation. The README says so.

6. The AI review was reading a lossy picture of the code

- **Found by:** driving the real UI with the Elevator problem. Two of the three AI findings were factually wrong: it said `ElevatorSystem` held a single `Elevator` (the field was a `List<Elevator>`) and that `stops` was a raw `TreeSet` (it was `TreeSet<Integer>`).
- **Cause:** the assistant's own prompt renderer printed the unwrapped type name, as the build spec describes for `FieldNode.type`, so the model could not see the generics. This was an assistant-introduced defect, not a model failure.
- **Decision:** keep the unwrapped type for relationships and add a separate declared type for the prompt and diagram. After the change the same design produced no such claims. Tests assert the prompt shows `List<Car>` and never includes method bodies.
- **Lesson recorded:** unit tests with a fake LLM could not have caught this; only a real review of a real design did.

7. A rule that rejected the right answer

- **Spec said:** ignore any method that repeats an entry-type method's name and parameter count when looking for who owns a capability.
- **Found by:** the same Elevator run, where `Elevator.step()` was rejected as the mover because `ElevatorSystem.step()` has the same name.
- **Decision:** the assistant narrowed it to the entry type's own public methods and added `pick`, `choose`, `nearest` and `serve` as dispatch hints. Two earlier tests were rewritten and three added. This is a deviation from the spec and needs the owner's confirmation.

8. A thread-safety bug found by the evaluation harness

- **Found by:** the live review check. Four designs reviewed in parallel; the parking designs came back with no findings at all, even the deliberately bad one.
- **Cause:** the assistant's `JavaCodeParser` held one `JavaParser` shared by every thread. Under concurrent use valid code was reported as a syntax error ("Index 2147483647 out of bounds"). In the real app this would show when two learners submit at once.
- **Decision:** one parser per thread. A test with 8 threads and 400 parses failed before the change and passes after.
- **Lesson recorded:** the single-threaded unit tests passed for the whole build; only running real work in parallel exposed it.

9. Evaluating the AI review, and who did the labelling

- **What the assistant did:** wrote six designs, labelled the expected decision-point approaches, and chose the pass thresholds in `docs/REVIEW_EVALUATION.md`.
- **Risk to check:** the same assistant built the platform and wrote its own exam. The numbers (94% decision-point match, 9% invented findings) show the loop behaves sensibly on those six designs, not that the review is accurate. The owner should replace or check the designs and labels.

What the AI did not do

- The automated tests never call the real Gemini service (scripted fake only). The live service was exercised by hand through the UI on all three problems, with one design each. That shows the flow works, not that the review is accurate.
- It did not compare the AI review with human-written judgments.
- It did not tune rule thresholds on real learner code.